Components Map

ProductHunt Partnership Scout — Mesa Cohort C4 · AI and its Applications · Final Examination, Problem 4

The problem in one paragraph

Kavya runs partnerships at a B2B SaaS company that makes async-standup software. Every morning she spends 30–40 minutes scrolling ProductHunt to find newly launched tools she could partner with, and still misses some. The first 7 days after a launch are the best time to contact a founder. She needs a tool that does the scrolling, scoring and first-draft thinking for her, on demand.

The solution in one sentence

Kavya clicks one button → the app reads this week's ProductHunt launches → ranks them by upvotes and how actively the makers reply → asks Gemini AI how each product could partner with hers → shows a ranked outreach list.

The big picture: Input → Processing → Output

1 · INPUT

What goes in

- **From Kavya:** categories to scout, a short description of her own product, how many products she wants
- **From ProductHunt (live):** every launch from the last 7 days, with upvotes, makers and comments

→

2 · PROCESSING

What happens to it

- Fetch & merge launches
- Count maker comments
- Score & rank
- Gemini AI judges fit and writes the outreach angle
- Save the scan

→

3 · OUTPUT

What comes out

- Ranked list: name, founder, one-line description, upvotes, days since launch, outreach angle
- Fit badge, score, maker comments
- CSV download and scan history

Where each part lives

PART	TOOL	PLAIN-ENGLISH JOB
Screen Kavya sees	Next.js website	The page with buttons, the product box and the results table
Brain / back-end	Vercel serverless function	Runs the 5 processing steps when the button is clicked
Live data	ProductHunt API	Source of all launch data — nothing is made up
AI	Gemini Flash	Reads each product and writes the angle (the only AI used)
Memory	Supabase database	Stores past scans so the page is never empty
Code home	GitHub → Vercel	Code is stored on GitHub; Vercel publishes it as the public website

PART 1

INPUT — what goes into the system

There are two kinds of input: things **Kavya types or clicks**, and things the app **fetches live** from ProductHunt.

A · Inputs from Kavya (the website)

INPUT	HOW SHE GIVES IT	EXAMPLE	WHY IT MATTERS
Categories	Click buttons	Project Management, HR Tech, Developer Productivity, Team Communication	Decides which parts of ProductHunt to search
Her product description	Type in a text box	"An async standup tool for remote teams…"	Lets the AI judge fit and write angles specific to her product
List size	Slider (5–15)	10	How many products to show
Go	Click "Scout this week's launches"	—	Runs everything on demand

B · Live data from ProductHunt

The app asks ProductHunt's official API (GraphQL) for launches posted in the last 7 days. It searches each category's matching ProductHunt topic:

KAVYA'S CATEGORY	PRODUCTHUNT TOPIC(S) SEARCHED
Project Management	`task-management`
HR Tech	`human-resources` , `hiring`
Developer Productivity	`developer-tools`
Team Communication	`remote-work` , `meetings`

For each launch the app receives:

DATA FIELD	USED FOR
Name, tagline, description, topics	Showing the product and letting Gemini understand it
Upvote count	Score (traction) and the "Upvotes" column
Launch date	"Days since launch" column
Makers (founders) — name and ID	"Founder" column and identifying maker comments
Comments and replies (top 20 products only)	Counting how often the makers reply

Real numbers from the live run: one scan reviewed **113** launches from the last 7 days across the four categories.

PART 2

PROCESSING — the 5 steps, one by one

All five steps run automatically inside one back-end function in about 25 seconds.

1 Fetch & merge

ProductHunt API

Ask ProductHunt for the best launches of the last 7 days in each topic (in parallel). Some products appear under several topics, so duplicates are removed. Keep the **20 strongest by upvotes** as the candidate pool.

IN
Topic names + "last 7 days"

OUT
~113 launches → 20 candidates

2 Measure maker responsiveness

ProductHunt API

For each of the 20 candidates, read the comments and count how many comments and replies were **written by the product's own makers**. A founder who replies a lot is easier to reach. (ProductHunt hides commenters' names from apps, so we match each comment's hidden user ID to the makers' IDs.)

IN
20 products' comments (first 20 comments + 5 replies each)

OUT
"Maker comments" number per product

If it can't be measured (some big-company launches list no public makers) the table shows "–" instead of a fake 0.

3 Score & rank

Plain code (no AI)

Each product gets a score from 0 to 100: 60% from upvotes (traction) and 40% from maker comments (responsiveness). Both are log-scaled against the best product in the pool, so one viral launch doesn't make everyone else look tiny.

```
score = 100 × ( 0.6 × log(1+upvotes) / log(1+best upvotes)
              + 0.4 × log(1+maker comments) / log(1+best maker comments) )
```

Worked example (real data). Best upvotes in pool = 548 (Rill Browser); best maker comments = 27 (Velozity).

PRODUCT	UPVOTES	MAKER COMMENTS	UPVOTE PART (×0.6)	MAKER PART (×0.4)	SCORE
Velozity	479	27	0.979 × 0.6 = 0.587	1.000 × 0.4 = 0.400	**98.7**
Rill Browser	548	19	1.000 × 0.6 = 0.600	0.899 × 0.4 = 0.360	**96.0**

Rill has more upvotes, but Velozity's makers answer far more comments, so Velozity ranks first — exactly the "responsive team" signal the brief asks for.

4 Judge fit & write the outreach angle

Gemini Flash (only AI used)

The 20 candidates and Kavya's product description are sent to Gemini Flash (in two small batches, in parallel). For every product Gemini returns three things:

GEMINI RETURNS	WHAT IT MEANS
One-liner	A plain, hype-free description (max ~18 words)
Outreach angle	Two sentences Kavya can use in her first message: a concrete way the two products could work together (integration, co-marketing, referral, audience swap)
Fit rating	**high** = same buyer and a natural integration · **medium** = overlapping audience, stretchy idea · **low** = different buyer, consumer product, big-company or infrastructure launch, or competitor

Rules given to Gemini: write proposals ("we could…"), never invent facts about Kavya's product, and never just say "no overlap".

Then the final list is chosen: the highest-scoring products that are *not* "low fit" come first; weak fits only fill empty slots. The order is always by the score from step 3.

IN
20 candidates + Kavya's product text

OUT
10 products (her chosen size) with one-liner, angle, fit

Reliability: Gemini's free tier limits requests per model, so the app automatically tries the next Flash model if one is full or busy. Results are cached, so repeating the same scan takes ~3 seconds and uses no AI quota.

5 Save the scan

Supabase (Postgres)

Every finished scan — Kavya's inputs and the ranked results — is stored in a database table. When the website opens, it shows the latest saved scan straight away, so there is always data on screen (this is the "works with data at submission" requirement).

IN
Finished ranked list

OUT
Saved row; shown on next page load; "Recent scans" list

What happens when something goes wrong

PROBLEM	WHAT THE APP DOES
A ProductHunt topic fails	The other topics still load; a warning is shown
Maker comments can't be measured	Shows "–" and scores on upvotes only — never a made-up number
Gemini model is busy or out of quota	Tries the next Flash model; if all fail, shows ProductHunt's own taglines with a visible warning
Database save fails	Results still appear; a warning says history wasn't saved
ProductHunt rate limit hit	Shows an error and keeps the last saved scan visible

PART 3

OUTPUT — what Kavya gets

A ranked table, best partner first. Here is **one real row** from the live app, and where each field comes from:

Rank	1 (highest score)
Product name	Velozity
Founder	Ishu Darshan
One-line description	A virtual multiplayer office unifying messaging, calls, tasks, calendars, and AI agents for remote teams.
Upvote count	479
Days since launch	5
Maker comments · Score · Fit	27 · 98.7 · high fit
Suggested outreach angle	We could explore integrating our async standup summaries directly into Velozity's unified team workspace. This would allow remote team managers to view daily progress and blockers alongside their active tasks and calls.

REQUIRED OUTPUT FIELD	WHERE IT COMES FROM
Product name	ProductHunt API (step 1)
Founder name	ProductHunt API — first public maker (step 1)
One-line description	Gemini Flash (step 4), falls back to ProductHunt tagline
Upvote count	ProductHunt API (step 1)
Days since launch	Calculated from the launch date (today − launch date)
Suggested outreach angle	Gemini Flash (step 4), using Kavya's product description

Extras Kavya also gets

- **Maker comments, score and fit badge** — so she can see *why* a product ranked where it did.
- **Export CSV** — download the list for a spreadsheet or CRM.
- **"Hide low-fit" switch** and a **Recent scans** list to revisit earlier results.
- **Link to each product's ProductHunt page** to start outreach.

Human in the loop: the outreach angles are AI suggestions. Kavya reviews them before sending anything to a founder.

Quick glossary

API – a way for one program to ask another for data · **Serverless function** – back-end code that runs only when called · **Maker** – a product's founder on ProductHunt · **Gemini Flash** – Google's fast AI model (the only AI used) · **Log-scaling** – compresses big numbers so one outlier doesn't dominate · **Cache** – saved answers reused to be faster and cheaper